Learn JavaScript: Beginner's Course

Lesson 3: Operators and Expressions

Operators and Expressions - Study Notes

Key Concepts

- **Arithmetic operators** (`+`, `-`, `*`, `/`, `%`, `**`) for numeric calculations.
- **Comparison operators** (`==`, `!=`, `<`, `>`, `<=`, `>=`) that return Boolean values.
- **Logical operators** (`and`, `or`, `not`) for combining or inverting Boolean expressions.
- **String concatenation** (`+`) and repetition (`*`) for building and manipulating text.
- **Operator precedence** and the use of parentheses to control evaluation order.

Summary

Operators are symbols that tell a programming language to perform specific actions on one or more operands (values or variables). In this lesson we explored three families of operators that are fundamental to writing expressions: arithmetic, comparison, and logical. Arithmetic operators let us do math—addition, subtraction, multiplication, division, modulus, and exponentiation. Comparison operators evaluate relationships between two values and produce a Boolean result (`True` or `False`), which is essential for decision making constructs like `if` statements. Logical operators let us combine or invert those Boolean results, enabling more complex conditions. Finally, we saw how the `+` operator can also concatenate strings, and how `*` can repeat a string a given number of times. Understanding operator precedence and how to use parentheses ensures that expressions are evaluated exactly as intended.

Important Points

- **Arithmetic**: `+` (add), `-` (subtract), `*` (multiply), `/` (float division), `//` (integer division), `%` (modulus), `**` (exponent).
- **Comparison**: `==` (equal), `!=` (not equal), `<` (less), `>` (greater), `<=` (less or equal), `>=` (greater or equal). All return `bool`.
- **Logical**: `and` (True if both operands True), `or` (True if at least one operand True), `not` (inverts Boolean).
- **String ops**: `+` joins two strings; `*` repeats a string `n` times (`"ha"*3 !' "hahaha"`).
- **Precedence (high !' low)**: exponentiation (`**`), unary `+`/`-`, multiplication/division/modulus (`* / // %`), addition/subtraction (`+ -`), comparisons, `not`, `and`, `or`. Parentheses override this order.
- Mixing types (e.g., adding a string to an integer) raises a `TypeError`; explicit conversion (`str()`, `int()`) is needed when required.

Examples

Example 1: Calculating the area of a circle

```python
import math
radius = 5
area = math.pi * radius ** 2   # ** has higher precedence than *
```

```
print(f"Area: {area:.2f}")    # Area: 78.54
```

Explanation: `radius ** 2` is evaluated first (exponentiation), then multiplied by `math.pi`. The result is formatted to two decimal places.

Example 2: Building a greeting with string concatenation and repetition

```python
name = "Ada"
greeting = "Hello, " + name + "! " * 3
print(greeting)   # Hello, Ada! Hello, Ada! Hello, Ada!
```

Explanation: `"Hello, " + name + "!"` creates `"Hello, Ada!"`. The `* 3` repeats that string three times, producing the final greeting.

Quick Review

1. What does the expression `7 % 3` evaluate to?
2. Which operator has higher precedence: `+` or `*`?
3. Write a Boolean expression that is `True` only when `x` is between 10 and 20 (inclusive).
4. What is the result of `"ab" * 4 + "c"`?
5. How would you force the addition `a + b` to happen before multiplication with `c` in the expression `a + b * c`?

Further Reading

- Operator precedence tables for your specific language (e.g., Python, JavaScript, Java).
- Type conversion and casting (`int()`, `float()`, `str()`).
- Bitwise operators (`&`, `|`, `^`, `<<`, `>>`) – a next step after mastering logical operators.
- Short circuit evaluation of `and` and `or`.
- Using expressions in control flow (`if`, `while`, list comprehensions).